
Plate-Local River Generation

Michael K. Davis III
mkd21@fsu.edu

June 8, 2026

Abstract

This work documents a deterministic procedural world-generation system that produces coherent continents and terrain-aware, downhill-flowing river networks without precomputing an entire world. Each tectonic plate is defined by a deterministic seed position that serves a dual purpose: it supports direct pointwise terrain evaluation through continuous plate-derived fields, and it supports the construction of a bounded geometric plate domain for river preprocessing. The plate-derived fields place landmasses, mountain ranges, plateaus, and island chains according to plate type, border distance, and drift direction. The geometric domain uses straight Voronoi-like borders rather than reproducing the noise-warped borders of the pointwise terrain fields. This deliberate approximation allows the river generator to process a bounded plate directly without sampling terrain to locate its borders. Pre-river terrain is sampled on a low-resolution grid within each domain. River paths descend from climate-appropriate highland source nodes until reaching a coastline, local minimum, or existing confluence. The resulting network is partitioned into geometric nearest-river regions, and river distance and interpolated river height are evaluated on demand for blending with the base terrain. River networks are generated and cached lazily per plate, allowing independent regional processing without a world-wide generation pass.

1 Introduction

Procedural, deterministic, locally generatable terrain is well suited to large worlds because the terrain can be generated in independent chunks that remain seamless with neighboring chunks. However, large-scale features such as continents and river networks may depend on terrain relationships that extend far beyond the chunk currently being generated.

This work presents a plate-local river generation method for deterministic procedural worlds. Tectonic plates are used as intermediate preprocessing domains: large enough to contain regional drainage structure, but small enough to generate independently. Each plate is represented both as continuous pointwise terrain fields and as a bounded geometric domain for river planning. River networks are generated lazily per plate and are queried during chunk generation to blend interpolated river height with locally evaluated terrain.

The key contributions of this work are:

- dual plate representation for terrain sampling and river preprocessing, described in Sections [2.2–2.4](#);
- lazy local river generation and caching, described in Section [2.7](#);
- nearest-river regions that accelerate river-segment lookup during chunk generation, described in Section [2.7](#).

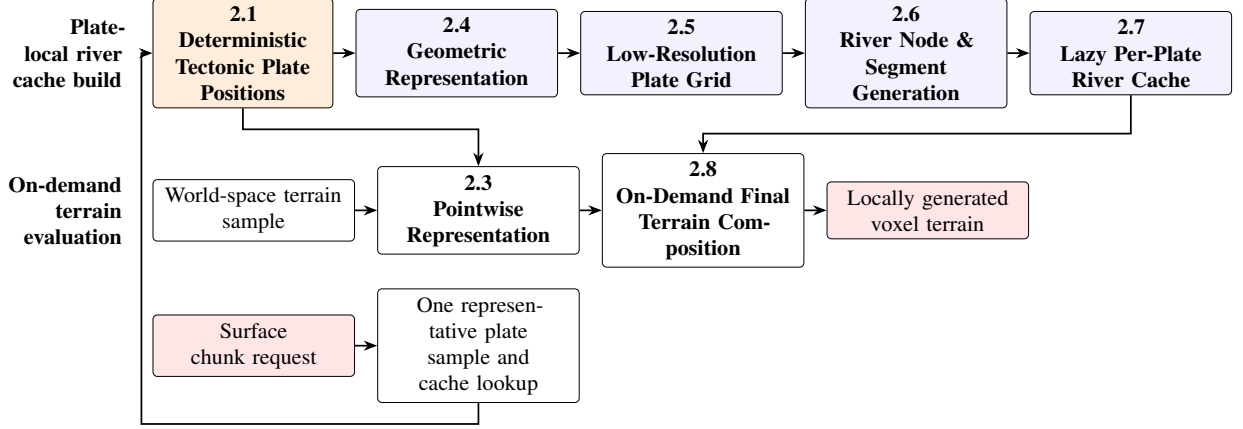


Figure 1: Pipeline overview, generalized for various world types—infinite or finite, and landscape structures—voxels or mesh.

2 Method

2.1 Deterministic Tectonic Plate Positions

Each tectonic plate is defined by a deterministic seed position. The same construction can be used for finite and infinite worlds: finite worlds use a bounded set of plate identifiers, while infinite worlds generate plate identifiers from lattice cells on demand. In both cases, terrain samples are assigned to the nearest deterministic seed position, producing Voronoi-like plate regions [1, 2].

For an infinite world, plate seeds may be generated as one deterministic point per integer lattice cell, following the same nearest-feature-point structure used by cellular texture bases [6],

$$\mathbf{c}_i = s_p (\mathbf{i} + \boldsymbol{\eta}(\mathbf{i})), \quad \mathbf{i} \in \mathbb{Z}^d, \quad (1)$$

where s_p is the plate scale, i is the plate identifier associated with integer lattice-cell coordinate $\mathbf{i}$, and $\boldsymbol{\eta}(\mathbf{i})$ is a deterministic hash offset. For a finite world, with a closed geometry and N plates, a golden-angle sequence gives approximately even seed directions [3, 4]:

$$\theta_i = i\pi(3 - \sqrt{5}), \quad y_i = 1 - \frac{2i+1}{N}, \quad r_i = \sqrt{1 - y_i^2}, \quad (2)$$

$$\mathbf{q}_i = (r_i \cos \theta_i, y_i, r_i \sin \theta_i), \quad \mathbf{c}_i = s_p \Pi_{\mathcal{S}}(\mathbf{q}_i) + \boldsymbol{\eta}_i, \quad 0 \leq i < N, \quad (3)$$

Here $\Pi_{\mathcal{S}}$ projects the unit direction onto the chosen finite world surface, and $\boldsymbol{\eta}_i$ is a deterministic jitter applied to the finite plate center.

2.2 Dual Tectonic Plate Representation

The tectonic seed positions are used in two forms: a *pointwise representation*, defined by continuous coordinate-based fields for terrain evaluation, and a *geometric representation*, defined by a bounded non-rasterized domain for regional river preprocessing. The same seed positions drive both local terrain fields and plate domains for drainage-network generation, allowing individual plates to be processed without rasterizing the surrounding world.

2.3 Pointwise Representation

During chunk generation, terrain is evaluated directly from world-space coordinates by comparing plate seed positions and deriving continuous scalar and vector weight fields from plate ownership, plate type, border distance, and drift direction. These weight fields control land, mountain, plateau, and island chain placement. See Figure 2(a). Noise warping and surface roughness are applied to produce more organic continent shapes [5]. See Figure 2(c).

Pointwise Plate. For a world-space point $\mathbf{x}$, the pointwise plate fields are evaluated in warped plate space,

$$\tilde{\mathbf{x}} = s_p \mathbf{x} + \mathbf{w}(\mathbf{x}), \quad (4)$$

where $\mathbf{w}$ is a deterministic noise warp. Note that $\mathbf{w}$ cannot be represented in the geometric representation of the same plate. Given deterministic plate centers $\mathbf{c}_i$, the owning plate is the nearest center in this space:

$$i = \arg \min_{j \in \mathcal{A}} \|\tilde{\mathbf{x}} - \mathbf{c}_j\|^2. \quad (5)$$

Here $\mathcal{A}$ is the set of possible plate identifiers. For a finite world, $\mathcal{A} = \{1, \dots, N\}$ and each identifier names one generated plate seed. For an infinite world, $\mathcal{A} = \mathbb{Z}^d$ and each identifier is an integer lattice-cell coordinate that generates one seed on demand. Plate fields are controlled by the distance margin between the owning plate and a neighboring plate:

$$d_j(\mathbf{x}) = \|\tilde{\mathbf{x}} - \mathbf{c}_j\| - \|\tilde{\mathbf{x}} - \mathbf{c}_i\|, \quad j \neq i. \quad (6)$$

Small values of d_j indicate proximity to the boundary between the owning plate and plate j . When a terrain feature depends on the nearest eligible neighboring plate, the margin is minimized over a selected neighbor set $\mathcal{N}$:

$$d_{\mathcal{N}}(\mathbf{x}) = \min_{j \in \mathcal{N}} d_j(\mathbf{x}). \quad (7)$$

Plate Drift. Each plate identifier is assigned a deterministic drift axis $\mathbf{a}_i$ by projecting a random vector into the tangent plane of the terrain domain. For a non-flat world, such as a finite one with a closed geometry, the plate center direction vector from the world origin provides a stable reference normal:

$$\mathbf{a}_i = \frac{\mathbf{c}_i \times \mathbf{r}_i}{\|\mathbf{c}_i \times \mathbf{r}_i\|}. \quad (8)$$

For a flat world, such as an infinite one, the constant reference normal $\mathbf{n}_0$ can be used instead:

$$\mathbf{a}_i = \frac{\mathbf{n}_0 \times \mathbf{r}_i}{\|\mathbf{n}_0 \times \mathbf{r}_i\|}. \quad (9)$$

The drift direction at the terrain sample is then

$$\ell(\mathbf{x}) = \frac{\mathbf{a}_i \times \mathbf{n}(\mathbf{x})}{\|\mathbf{a}_i \times \mathbf{n}(\mathbf{x})\|}, \quad (10)$$

where $\mathbf{r}_i$ is the deterministic random vector assigned to plate identifier i , $\mathbf{n}(\mathbf{x})$ is the local surface normal, and $\ell(\mathbf{x})$ is the local tangent drift direction used at the terrain sample. The scalar drift field, $\rho(\mathbf{x})$, measures signed progress along the local drift direction:

$$\rho(\mathbf{x}) = (\tilde{\mathbf{x}} - \mathbf{c}_i) \cdot \ell(\mathbf{x}). \quad (11)$$

Weights. Land, island, and mountain feature weights use the same normalized beta-shaped border profile. For feature k with amplitude A_k , width ω_k , interior exponent p_k , border exponent q_k , and plate-coverage parameter γ_k ,

$$C_k = \left[\left(\frac{p_k}{p_k + q_k} \right)^{p_k} \left(\frac{q_k}{p_k + q_k} \right)^{q_k} \right]^{-1}, \quad (12)$$

$$D_k(\mathbf{x}) = \max \left(0, \frac{\rho(\mathbf{x}) + 2\gamma_k - 1}{2\gamma_k} \right), \quad (13)$$

$$W_k(\mathbf{x}) = \begin{cases} A_k C_k D_k(\mathbf{x}) \left(\frac{d_{\mathcal{N}_k}(\mathbf{x})}{\omega_k} \right)^{p_k} \left(1 - \frac{d_{\mathcal{N}_k}(\mathbf{x})}{\omega_k} \right)^{q_k}, & \text{if within the feature width,} \\ 0, & \text{if outside the feature width.} \end{cases} \quad (14)$$

The coefficient C_k normalizes the profile before amplitude scaling, and D_k gates the feature by the drift field.

Plateaus are treated separately because they arise from converging continental plates rather than a single boundary-distance profile. For a nearby continental pair (i, j) , let $\hat{\mathbf{u}}_{ij}(\mathbf{x})$ denote the unit separation direction from plate i to plate j after projection into the local tangent plane. The closing response is measured by projecting each plate's tangent drift onto this direction:

$$\kappa_{i \rightarrow j}(\mathbf{x}) = \begin{cases} (\mathbf{a}_i \times \mathbf{n}(\mathbf{x})) \cdot \hat{\mathbf{u}}_{ij}(\mathbf{x}), & \text{if positive,} \\ 0, & \text{if zero or negative,} \end{cases} \quad (15)$$

$$\kappa_{j \rightarrow i}(\mathbf{x}) = \begin{cases} -(\mathbf{a}_j \times \mathbf{n}(\mathbf{x})) \cdot \hat{\mathbf{u}}_{ij}(\mathbf{x}), & \text{if positive,} \\ 0, & \text{if zero or negative.} \end{cases} \quad (16)$$

These detect colliding plates. The weight itself is constructed as,

$$R_{\text{plateau}}(\mathbf{x}) = O(\mathbf{x}) \max_{(i,j) \in \mathcal{C}} B_{ij}(\mathbf{x}) N_i(\mathbf{x}) \kappa_{i \rightarrow j}(\mathbf{x}) \kappa_{j \rightarrow i}(\mathbf{x}) S\left(\frac{\kappa_{i \rightarrow j}(\mathbf{x})}{\eta}\right) S\left(\frac{\kappa_{j \rightarrow i}(\mathbf{x})}{\eta}\right), \quad (17)$$

$$W_{\text{plateau}}(\mathbf{x}) = A_{\text{plateau}} S(g_{\text{plateau}} R_{\text{plateau}}(\mathbf{x})) (A_{\text{land}} - W_{\text{land}}(\mathbf{x})) (A_{\text{mountain}} - W_{\text{mountain}}(\mathbf{x})). \quad (18)$$

Here $\mathcal{C}$ is the set of nearby continental plate pairs, B_{ij} fades with the distance between the plate centers, N_i fades with the distance from the owning plate, O suppresses plateau formation near oceanic boundaries, η is the closing fade width, g_{plateau} controls the plateau response gain, and S is a smoothstep function. The final plateau weight W_{plateau} is an amplitude-scaled smoothstep response to the closing response R_{plateau} , suppressed where the land and mountain fields already dominate.

2.4 Geometric Representation

The entire plate domain can be constructed geometrically from nearby seed positions without pointwise terrain sampling. Because the noise warping applied to the weight fields cannot be reproduced geometrically, this domain is a straight-bordered approximation of the true plate footprint—close enough for river generation purposes.

The geometric plate domain is initialized as a bounded volume B_i around the plate center and then clipped by the bisector planes between $\mathbf{c}_i$ and $\mathbf{c}_j$:

$$\Omega_i = B_i \cap \bigcap_{j \neq i} \{ \mathbf{p} : (\mathbf{p} - \mathbf{m}_{ij}) \cdot \mathbf{n}_{ij} \leq 0 \}, \quad (19)$$

where

$$\mathbf{m}_{ij} = \frac{1}{2} (\mathbf{c}_i + \mathbf{c}_j), \quad (20)$$

$$\mathbf{n}_{ij} = \mathbf{c}_j - \mathbf{c}_i. \quad (21)$$

Note that $j \in \mathcal{N}_i$, where $\mathcal{N}_i$ is finite. For a world surface $\mathcal{S}$, the plate-local preprocessing domain is the surface cross section of this clipped volume,

$$\Omega_i^{\mathcal{S}} = \Omega_i \cap \mathcal{S}. \quad (22)$$

Recall that this geometric representation is not identical to the pointwise representation due to the warp term $\mathbf{w}$ in Equation (4).

2.5 Low-Resolution Plate Grid

The river generator samples pre-river terrain using the weight fields on a low-resolution grid restricted to the geometric representation domain, providing approximate terrain elevation and slope for river path planning.

For each accepted grid sample (u, v) on plate i , the sample position lies in the geometric surface domain and stores the pre-river terrain height:

$$\begin{aligned} \mathbf{x}_{uv} &\in \Omega_i^{\mathcal{S}}, \\ h_{uv} &= A_{\text{pre}} H_{\text{pre}}(\mathbf{x}_{uv}), \end{aligned} \quad (23)$$

where $A_{\text{pre}} < 1$ is a pre-river height attenuation factor. By reducing the terrain height used for river planning, the subsequently enforced river elevations are biased below the surrounding terrain surface, producing valley-like depressions after composition. A_{pre} may be defined to attenuate land, mountain, plateau, and island features differently.

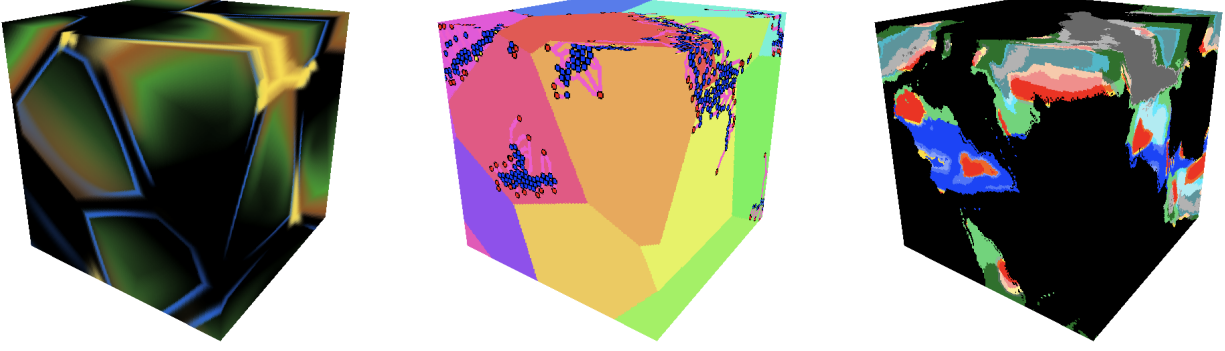
2.6 River Node & Segment Generation

River source nodes are selected from grid points of sufficient elevation. Segments are traced downhill along the grid until reaching sea level, a local minimum, or an existing river segment, producing outlets, lakes, or confluences respectively.

Note that the algorithm for placing nodes and segments is arbitrary and can be modified or replaced without affecting the rest of the pipeline. However, to avoid blending discontinuities during final terrain composition, nodes and segments should not be placed near plate borders. The river blending width, ω_{river} (see Section 2.8), defines a proximity mask for removing or preventing node placement near these borders. Because river-terrain blending does not extend beyond ω_{river} from a given river, this constraint prevents river blending from crossing plate borders and introducing discontinuities.

2.7 Lazy Per-Plate River Cache

A plate’s river network is generated the first time a chunk samples terrain from that plate and finds the cache absent. The cache stores river nodes, segments, and elevations, along with a non-rasterized geometric partitioning of the plate domain into nearest-river regions. These nearest-river regions allow chunk generation



(a) Plate-derived continent fields

(b) Plate-local river networks

(c) Final continental output

Figure 2: Global views of the continent and river systems on a cube-shaped planet. **(a)** Plate-derived weight fields place land (green), mountains (brown), island chains (blue), and plateaus (yellow). **(b)** Each river network is generated inside a straight-bordered geometric plate domain. Blue dots mark river sources, red dots mark sea-level outlets, yellow dots mark lakes at local minima, and pink lines mark river segments. **(c)** Final landmasses include continental warping, surface roughness, and climate classification.

to locate the nearest river segment efficiently without searching the full network. Because rivers naturally should not cross tectonic boundaries—where plateaus and oceans generate within this system—each plate’s network can be generated and cached in isolation without triggering generation of neighboring plates.

For a terrain sample on plate i , let $\mathcal{R}_i$ be the cached set of river segments for that plate. The nearest segment and its distance are

$$\begin{aligned} s^*(\mathbf{x}) &= \arg \min_{s \in \mathcal{R}_i} \text{dist}(\mathbf{x}, s), \\ \delta_{\text{river}}(\mathbf{x}) &= \text{dist}(\mathbf{x}, s^*(\mathbf{x})). \end{aligned} \quad (24)$$

2.8 On-Demand Final Terrain Composition

During chunk generation, plate weight fields are sampled, distance to the nearest river segment is computed, and river elevation is interpolated between nodes. The river distance is converted into a normalized terrain blend weight before mixing the local terrain height with the river height:

$$\begin{aligned} \lambda_{\text{river}}(\mathbf{x}) &= \left[\min \left(1, \frac{\delta_{\text{river}}(\mathbf{x})}{\omega_{\text{river}}} \right) \right]^{\alpha_{\text{river}}}, \\ H_{\text{final}}(\mathbf{x}) &= \lambda_{\text{river}}(\mathbf{x}) H_{\text{pre}}(\mathbf{x}) + (1 - \lambda_{\text{river}}(\mathbf{x})) H_{\text{river}}(\mathbf{x}). \end{aligned} \quad (25)$$

Here ω_{river} is the river blending width and α_{river} controls the blend falloff. Near the river, λ_{river} approaches zero and the interpolated river height dominates; away from the river, λ_{river} approaches one and the locally generated terrain is preserved.

3 Results

A video demonstration is available on YouTube: youtube.com/watch?v=4Ai_13znvvgg.

The resulting terrain is generated locally. Figure 2 shows continental and river output from the current implementation, and Table 1 reports measured performance on representative hardware.

The implementation is written in Python and rendered using OpenGL and PyGame. No game engine is used. Hot terrain, physics, chunk, and query functions use Numba `@njit` to compile to machine code [7]. Additional algorithm optimizations are used elsewhere.

Plate	Cache (s)	Memory (MiB)	Segments
1	5.428	18.356	151
2	1.687	11.125	76
3	1.862	10.943	87
4	6.191	17.912	149
5	2.761	16.249	107
Mean	3.586	14.917	114.0

Table 1: Per-plate river cache measurements taken for five measured plate caches. These were collected on an Apple M4 Max with 48 GB RAM, running macOS 15.7.3. The low-resolution river grid resolution is 200 per plate. The max count for river source nodes is 50 per plate.

4 License

This work is licensed under the [Creative Commons Attribution 4.0 International License \(CC BY 4.0\)](https://creativecommons.org/licenses/by/4.0/).

References

- [1] F. Aurenhammer, "Voronoi diagrams—a survey of a fundamental geometric data structure," *ACM Computing Surveys*, vol. 23, no. 3, pp. 345–405, 1991.
- [2] A. Okabe, B. Boots, K. Sugihara, and S. N. Chiu, *Spatial Tessellations: Concepts and Applications of Voronoi Diagrams*, 2nd ed. Wiley, 2000.
- [3] E. B. Saff and A. B. J. Kuijlaars, "Distributing many points on a sphere," *The Mathematical Intelligencer*, vol. 19, no. 1, pp. 5–11, 1997.
- [4] R. Swinbank and R. Purser, "Fibonacci grids: A novel approach to global modelling," *Quarterly Journal of the Royal Meteorological Society*, vol. 132, no. 619, pp. 1769–1793, 2006.
- [5] K. Perlin, "An image synthesizer," *ACM SIGGRAPH Computer Graphics*, vol. 19, no. 3, pp. 287–296, 1985.
- [6] S. Worley, "A cellular texture basis function," *Proceedings of SIGGRAPH 1996*, pp. 291–294, 1996.
- [7] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015.